

第八讲 格式化输出

一、格式化输出

● 格式化输出函数参数由一个格式字符串和可变数目的参数构成

→ 格式化字符串提供了一组可以由格式化输出函数解释执行的指令

→ 用户可以通过控制格式字符串的内容来控制格式化输出函数的执行

● 变参函数在 C 语言中实现的局限性导致格式化输出函数的使用中容易产生漏洞!

```
#include <stdio.h>
#include <string.h>
```

```
void usage(char *pname) {
    char usageStr[1024];
    snprintf(usageStr, 1024,
        "Usage: %s <target>\n",
        pname);
    printf(usageStr);
}
```

pname的运行时值将替换格式字符串中的%d从而构造用法usage字符串

调用printf()函数输出usage信息

```
int main(int argc, char *
    if (argc < 2) {
        usage(argv[0]);
        exit(-1);
    }
}
```

程序的实际名字由用户输入 (argv[0]) 并作为参数传递给usage()函数

北京邮电大学

二、变参函数

变参函数既可以通过 UNIX System V 实现，也可以通过 ANSI C 实现。两种方式都要求变参函数的用户不违反开发者提供的契约。

1、ANSI C 标准参数

在 ANSI C 的标准参数方式（也称为 stdargs）中，变参函数是通过使用一个部分参数列表后跟一个省略号进行声明的。

若要调用一个变参函数，仅需指定该次调用中所需数目的参数即可：`average(3, 5, 8, -1);`

使用 stdargs 实现 `average()` 函数：

```
int average(int first, ...) {
    int count = 0, sum = 0, i = first;
    va_list marker;
    va_start(marker, first);
    while (i != -1) {
        sum += i;
        count++;
        i = va_arg(marker, int);
    }
    va_end(marker);
    return(sum ? (sum / count) : 0);
}
```

变参函数 `average()` 接受一个独立的定参，其后跟着一个变参列表。

对该变参列表中的参数不会执行任何类型检查。

省略号之前通常有一个或多个定参，而省略号则必须出现在参数列表的最后。

ANSI C 为了实现变参函数所提供的宏，`va_start()`、`va_arg()`和
`va_end()` 的定义。

这些定义在头文件 `stdarg.h` 中的宏，全部作用于 `va_list` 数据类型
以及使用 `va_list` 类型声明的参数列表之上。

2、可变参数宏定义示例

```
● #define _ADDRESSOF(v) (&(v))
● #define _INTSIZEOF(n) \
● ((sizeof(n)+sizeof(int)-1) & ~(sizeof(int)-1))
● typedef char *va_list;

● #define va_start(ap,v)
  (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v))
● #define va_arg(ap,t) (*(t *)((ap+=_INTSIZEOF(t))-
  _INTSIZEOF(t)))
● #define va_end(ap) (ap = (va_list)0)

#define _ADDRESSOF(v) (&(v))
#define _INTSIZEOF(n) \
((sizeof(n)+sizeof(int)-1) &
~(sizeof(int)-1))

typedef char *va_list;

#define va_start(ap,v)
  (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v))
#define va_arg(ap,t) (*(t
  *)((ap+=_INTSIZEOF(t))-_INTSIZEOF(t)))
#define va_end(ap) (ap = (va_list)0)
```

变量`maker`被声明为`va_list`类型

在使用变量`marker`之前，首先必须调用`va_start()`对它进行初始化

第4行调用了`va_start()`并且传递参数`marker`以及最后一个定参（`first`）

宏 `va_arg()` 需要一个已初始化的 `va_list` 和下一个参数的类型。

这个宏可以根据这些信息返回下一个参数的值，并且相应地递增
参数指针。

`average()` 第 8 行调用 `va_arg()` 宏，通过循环，获取第 2 个直至最后
一个参数。

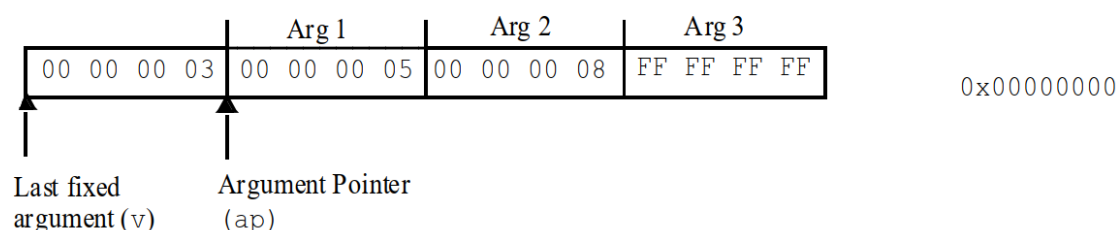
最后，在函数返回之前，调用 `va_end()` 来执行清理工作。

参数列表的终止条件是函数的实现者和使用者之间的一个契约。

在函数 `average()` 的实现中，变参列表的终止是由一个值为 -1 的参数所指定的。

如果程序员在调用该函数时忘记传入这个特殊的参数，则函数将继续处理下一个参数，直到遇到 -1 或者发生异常为止。

3、采用字符指针定义的 `va_list` 类型



图示例了在这些系统中调用函数 `average(3,5,8,-1)` 时，参数被如何按序安排在栈上

在用 `va_start()` 初始化字符指针后，字符指针指向最后一个定参之后的那个参数。

`va_start()` 宏将该参数的（类型）大小加上最后一个定参的地址。

当 `va_start()` 返回时，`va_list` 将指向第一个可选参数的地址。

并不是所有系统都把 `va_list` 类型定义成字符指针。

一些系统将其定义成指针数组，另外一些系统则把它放在寄存器中作为参数传递。

当参数在寄存器中传递时，`va_start()` 可能不得不为它们分配内存空间。

宏 `va_end()` 就被用来释放分配的内存空间。

4、`__cdecl`、`__stdcall` 和 `__fastcall`

	<code>__stdcall</code>	<code>__cdecl</code>	<code>__fastcall</code>
参数传递方式	右->左 压栈	右->左 压栈	左边开始的两个不大于 4 字节 (DWORD) 的参数分别放在 ECX 和 EDI 寄存器，其余的参数仍旧自右向左压栈传送
清理栈方	被调用函数清理 (即函数自己清理)，多数情况使用这个	调用者清理	被调用者清理栈
适用场合	Win API	c/C++ MFC 默认方式，可变参数的时候使用	速度快
C 编译修饰约定 (它们均不改变输出函数名中的字符大小写)	约定在输出函数名前加上一个下划线前缀，后面加上一个“@”符号和其参数的字节数，格式为 <code>_functionname@number</code>	约定仅在输出函数名前加上一个下划线前缀，格式为 <code>_functionname</code>	调用约定在输出函数名前加上一个“@”符号，后面也是一个“@”符号和其参数的字节数，格式为 <code>@functionname@number</code> 。
C++ 修饰	见下面的“四、名字修饰约定”		

(1) `__cdecl`

所有的参数都逆序压入栈中。

`__cdecl` 调用约定要求每一个函数调用都包含栈清理代码。

因为每次调用者都会对栈进行清空，所以这个调用约定支持对变参函数的使用。

这成了支持变参函数的必备条件，因为编译器不可能在没有检查实际调用的情况下确定到底向函数中传入了几个参数。

(2) __stdcall

__stdcall 调用约定用于调用 Win32 API 函数。

这个约定不支持变参函数，因为它是由被调用函数来执行清空栈操作的。

这就意味着当生成代码从栈中弹出实参时，编译器无法事先确定将有多少个参数被传递进函数中。

(3) __fastcall

_fastcall 调用约定会尽可能地将函数的参数放在寄存器中传递。

前两个双字或更小的参数将被放入寄存器 ecx 和 edx 中。

其余的参数按从右至左的原则传递。

被调用的函数将参数从栈中弹出。

5、使用 varargs 实现的 average()函数

```
int average(va_alist) va_dcl {  
    int i, count, sum;  
    va_list marker;  
    va_start(marker);  
    for (sum = count = 0;  
        (i = va_arg(marker, int)) != -1;  
        count++)  
        sum += i;  
    va_end(marker);  
    return (sum ? (sum / count) : 0);  
}
```

Unix System V的宏
(定义于varargs.h中)
的运作方式稍微不同
于ANSI标准参数

6、格式化字符串实现示例

```
#include "stdio.h"
#include "stdlib.h"
void myprintf(char* fmt, ...) //一个简单的类似于printf的实现, //参数必须都是int 类型
{
    char* pArg=NULL; //等价于原来的va_list
    char c;

    pArg = (char*) &fmt; //注意不要写成p = fmt !!因为这里要对//参数取址, 而不是取值
    pArg += sizeof(fmt); //等价于原来的va_start

    do
    {
        c =*fmt;
        if (c != '%')
        {
            putchar(c); //照原样输出字符
        }
        else
        {
            //按格式字符输出数据
            switch(*++fmt)
            {
                case 'd':
                    printf( "%d ",*((int*)pArg));
                    break;
                case 'x':
                    printf( "%#x ",*((int*)pArg));
                    break;
                default:
                    break;
            }
            pArg += sizeof(int); //等价于原来的va_arg
        }
        ++fmt;
    }while (*fmt != '\0');
    pArg = NULL; //等价于va_end
    return;
}
```

三、格式化输出函数

1、格式化输出函数

fprintf()按照格式字符串的内容将输出写入流中。

流、格式字符串和变参列表一起作为参数提供给函数。

printf()等同于 fprintf(), 除了前者假定输出流为 stdout 外。

sprintf()等同于 fprintf(), 但是输出不是写入流而是写入数组中。

snprintf()等同于 sprintf(), 但是它指定了可写入字符的最大值 n。

当 n 非零时, 输出的字符超过“n-1”的部分会被舍弃而不会写入数组中。并且, 在写入数组的字符末尾会添加一个空字符。

2、相同的函数

fprintf()、vfprintf()、vsprintf()、vsnprintf()、fprintf()、printf()、
sprintf()、snprintf()

3、syslog().

● 另外一个名为 syslog() 的格式化输出函数并没有定义在 C99 规范中，但是包含在 SUSv2 中。

● 这个函数接受一个优先级参数、一个格式规范以及该格式所需的任意参数，并且在系统的日志记录（syslogd）中生成一条日志消息。

● syslog() 函数：

→ 最先出现在 BSD 4.2

→ Linux and UNIX 支持

→ 未得到 Windows 系统的支持

4、格式字符串

● 格式字符串是由普通字符（包括%）和转换规范构成的字符序列。

● 普通字符被原封不动地复制到输出流中。

● 转换规范根据与实参对应的转换指示符对其进行转换，然后将结果写入输出流中。

● 转换规范通常以%开始按照从左向右的顺序解释。

● 当参数过多时，多余的将被忽略。

● 而当参数不足时，则结果是未定义的。

● 一个转换规范组成：

→ 可选域:

➤ 标志、宽度、精度以及长度修饰符

→ 必需域:

➤ 转换指示符, 按照下面的格式

`%[flags] [width] [.precision] [{lengthmodifier}] conversion-specifier.`

● 例如 `%-10.8ld`:

→ - 是标志位,

→ 10 代表宽度,

→ 8 代表精度,

→ l 是长度修饰符,

→ d 是转换指示符

● 这个转换规范将一个 `long int` 型的参数按照十进制格式打印, 在一个最小宽度为 10 个字符的域中保持最少 8 位左对齐。

● 最简单的转换规范仅仅包含一个 `%` 和一个转换指示符 (例如 `%s`)。

5、转换指示符

● 转换指示符用来指示所应用的转换类型。

● 它是唯一必需的格式域, 出现在任意可选格式域之后。

● 标志位用来调整输出和打印的符号、空白、小数点、八进制和十六进制前缀等。

● 一个格式规范中可能包含一个或多个标志。

6、宽度

宽度是一个用来指定输出字符的最小个数的十进制非负整数。如果输出的字符个数比指定的宽度小，就用空白字符补足。

如果指定的宽度较小也不会引起输出域的截断。如果转换的结果比域宽大，则域会被扩展以容纳转换结果。

如果使用星号 (*) 来指定宽度，则宽度将由参数列表中的一个 int 型的值提供。

在参数列表中，宽度参数必须置于被格式化的值之前。

7、精度

● 精度是用来指示打印字符个数、小数位数或者有效数字个数的非负十进制整数。

● 精度域可能会引起输出的截断或浮点值的舍入。

● 如果精度域是一个星号 (*), 那么它的值就由参数列表中的一个 int 参数提供。

● 在参数列表中，精度参数必须置于被格式化的值之前。

8、限制

(1) GCC3.2.2

GCC3.2.2 中的格式化输出函数可以处理最大为 INT_MAX (在 IA_32 上是 2,147,483,647) 的宽度和精度域。

格式化输出函数还以 int 的形式保持和返回被输出字符的总个数。

甚至当这个总个数的值超过了 INT_MAX 时它仍然会继续增加，从而引起带符号的整数溢出，结果得到一个带符号的负值。

如果按照无符号数来解释的话，那么直到发生无符号整型溢出之前这个数都是准确的。

(2) Visual C++ .NET

Visual C++ .NET 中各个格式化输出函数共享一个通用的格式字符串规范定义。

格式字符串通过一个名为 `_output()` 的共用函数进行解释。

`_output()` 基于从格式字符串中读出的字符和当前的状态来解析格式字符串并决定应该执行的动作。

`_output()` 函数使用一个带符号整数来存储宽度并且允许宽度值大至 `INT_MAX`。

因为 `_output()` 函数未对带符号整数溢出进行检测和处理，因此超过 `INT_MAX` 的值可能会导致非预期的结果。

`_output()` 函数同样使用带符号整数来存储精度，但它同时还使用一个 512 字符的转换缓冲区。

当这个值为负数时 `_output()` 主循环将会退出，从而值就被限制在 `INT_MAX+1` 和 `UINT_MAX` 之间了。

9、长度修饰符

Visual C++ 不支持 C99 中的长度修饰符 `h`, `hh`, `l`, 和 `ll`。

它提供的 `I32` 长度修饰符与长度修饰符 `I` 的功能完全一致，而 `I64` 长度修饰符则与长度修饰符 `II` 的作用相似。

`I64` 可以用于打印 `long long int` 所表示的所有值，不过当使用 `n` 转换指示符时只写出 32 位。

四、对于格式化输出函数的漏洞利用

见 PPT

五、栈随机化

1、栈随机化

● 在 Linux 下，栈的起始地址为 0xC0000000 并且朝低内存方向增长。

● 极少数 Linux 栈地址中包含空字节，从而容易使它们被插入格式字符串。

● 许多 Linux 变体中包含有某种栈随机化机制。

● 这种机制使得很难预测栈上信息的位置，包括返回地址和自动变量的位置，这是通过向栈中插入随机的间隙实现的。

2、阻碍栈随机化

● 尽管栈随机化加大了漏洞利用的难度，但它并不能完全阻止这种情况的发生。举个例子，上一节中展示的格式字符串漏洞利用需要这样的一些值：

→ 要覆写的地址，

→ shell code 的地址，

→ 参数指针和格式字符串起始地址之间的距离，

→ 在第一个转换规范%u 之前格式化输出函数已经写入的字节数。

(1) 待覆写地址

● 可以覆写在程序正常执行中、控制权将被转移到的函数的 GOT 入口或其他地址。

● 覆写 GOT 入口的优势在于它独立于诸如栈和堆这样的系统变量。

(2) Shellcode 的地址

● 基于 Windows 的利用中假设向栈的自动变量中插入了一段 shellcode。

● 对于实现栈随机化的系统，想找到这个地址很困难。

● 然而 shellcode 同样可以插入到数据段或堆上的变量中，这就比较容易找到了。

(3) 距离

● 攻击者必须确定参数指针和格式字符串的起始位置在栈中的距离。

● 它们之间的相对距离却是不变的。

● 计算参数指针与格式字符串的起始位置之间的距离并且插入所需数目的 %x 格式转换规范并不难做到。

(4) 以双字的格式写地址

● 基于 Windows 的利用将一个 shellcode 的地址分成四次每次写入一个字节，各次调用之间对地址进行递增。

● 如果由于对齐的要求或者其他原因造成这种操作不可能，仍然可以通过向地址中一次写入一个字甚至全部内容来实现。

3、Linux 利用变体

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
       "\x90\x09\x09\x09\x09\x09/bin/sh";

5.     int i;
6.     unsigned char format_str[1024];

7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");

11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }

    /* code to write address goes here */

14.    printf(format_str);
15.    exit(0);
16. }

```

静态变量向数据段中插入一个shellcode

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
       "\x90\x09\x09\x09\x09\x09/bin/sh";

5.     int i;
6.     unsigned char format_str[1024];

7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");

11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }

    /* code to write address goes here */

14.    printf(format_str);
15.    exit(0);
16. }

```

exit()函数的GOT入口的地址被连接到格式字符串

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
        "\x90\x09\x09\x09\x09\x09/bin/sh";

5.     int i;
6.     unsigned char format_str[1024];

7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");

11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }

    /* code to write address goes here */

14.    printf(format_str);
15.    exit(0);
16. }

```

在第10行中，该地址
被+ 2后也
连接到格式字符串

```

1. #include <stdio.h>
2. #include <string.h>

3. int main(int argc, char * argv[]) {

4.     static unsigned char shellcode[1024] =
        "\x90\x09\x09\x09\x09\x09/bin/sh";

5.     int i;
6.     unsigned char format_str[1024];

7.     strcpy(format_str, "\xaa\xaa\xaa\xaa");
8.     strcat(format_str, "\xb4\x9b\x04\x08");
9.     strcat(format_str, "\xcc\xcc\xcc\xcc");
10.    strcat(format_str, "\xb6\x9b\x04\x08");

11.    for (i=0; i < 3; i++) {
12.        strcat(format_str, "%x");
13.    }

    /* code to write address goes here */

14.    printf(format_str);
15.    exit(0);
16. }

```

当调用exit()终止程序时，控制权会
移交给shellcode

4、直接参数存取

● Single UNIX 规范[IEEE 04]允许转换被应用于参数列表中的格式之后的第 n 个参数上，而不是应用到下一个未使用的参数上。

● 转换指示符%将被序列所代替，

● %n\$,

● 其中 n 是一个 1 到 {NL_ARGMAX} 范围内的十进制整数，它指定了参数的位置。

格式既可以包含数字式，也可以包含非数字式的参数转换规范，但不允许二者同时出现。

- 数字式的:- %n\$ and *m\$
- 非数字式的:- % and *

%%与%n\$混合使用是个例外。

在一个格式字符串中混用数字式和非数字式参数规范会导致未定义的结果。

● 当使用数字式参数规范时，要想指定第 n 个参数，格式

● 字符串中所有从第一个到第 n-1 个前导参数都要被指定。

● 在包含有如%n\$形式的转换规范的格式字符串中，参数列表中的数字式参数可视需要被从格式字符串中引用多次。

● 1. int i, j, k = 0;

● 2. printf(

● "%4\$5u%3\$n%5\$5u%2\$n%6\$5u%1\$n\n",

● &k, &j, &i, 5, 6, 7

●);

● 3. printf("i = %d, j = %d, k = %d\n", i, j, k);

● Output:

● 5 6 7

● i = 5, j = 10, k = 15

第一个转换规范,%4\$5u获得第四个参数（即常量5），并将输出格式化为无符号的十进制整数，宽度为5。


```
1. int i, j, k = 0;
```

```
2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);
```

第二个转换规范%3\$n，将当前输出计数器的值（5）写到第三个参数(&i)所指定的地址。

```
3. printf("i = %d, j = %d, k = %d\n", i, j, k);
```

Output:

```
    5    6    7
i = 5, j = 10, k = 15
```

```
1. int i, j, k = 0;
```

```
2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);
```

第2行对printf()函数的调用导致以5个字符的列宽将值5、6、7打印出来。

```
3. printf("i = %d, j = %d, k = %d\n", i, j, k);
```

Output:

```
    5    6    7
i = 5, j = 10, k = 15
```

```
1. int i, j, k = 0;
```

```
2. printf(
    "%4$5u%3$n%5$5u%2$n%6$5u%1$n\n",
    &k, &j, &i, 5, 6, 7
);
```

```
3. printf("i = %d, j = %d, k = %d\n", i, j, k);
```

Output:

```
    5    6    7
i = 5, j = 10, k = 15
```

第3行调用的printf()打印出赋给变量i, j, 和k的值, 这些值代表了在上一次调用printf()的基础上输出计数器的增加值。

● 转换规范%n\$中的参数数字 n 必须是这样的一个整数值：其值必须介于 1 和提供给函数调用的参数的最大数目之间。

● 在 GCC 中，运行时真正有效的值可以通过 `sysconf()` 获得：`int max_value = sysconf(_SC_NL_ARGMAX);`

● 有些系统（如 System V）有一个下限值，如 9。在 GNU C 库中不存在实际的限制。Red Hat 9 Linux 中该最大值是 4096。

六、缓解策略

1、动态格式字符串

```
#include <stdio.h>
#include <string.h>

int main(int argc, char * argv[]) {
    int x, y;
    static char format[256] = "%d * %d = ";

    x = atoi(argv[1]);
    y = atoi(argv[2]);

    if (strcmp(argv[3], "hex")
        strcat(format, "0x%x\n");
    }
    else {
        strcat(format, "%d\n");
    }
    printf(format, x, y, x * y);

    exit(0);
}
```

这个程序是可以免于受到格式字符串利用的威胁的。

北京邮电大学

2、限制字节写入

● 缓冲区溢出可以通过严格控制这些函数写入的字节数来避免。

● 写入的字节数可以通过指定一个精度域作为 `%s` 转换规范的一部分进行控制。

● 例如，不使用 `sprintf(buffer, "Wrong command:%s\n", user);` 而是使用 `sprintf(buffer, "Wrong command:%.495s\n", user);`

● 精度域指定了针对 `%s` 转换所要写入的最大字节数。

● 在这个例子中：

→ 静态字符串“贡献”了 17 个字节。

→ 精度域为 495 确保结果字符串可以适合于 512 字节的缓冲。

● 另一种方式是使用更安全版本的格式化输出库函数，它们不容易产生缓冲区溢出问题。

● 例如

→ `snprintf()` 比 `sprintf()` 更好

→ `vsnprintf()` 替代 `vsprintf()`

● 这些函数指定了写入的最大字节数（包括末尾的空字节在内）。

● 函数 `asprintf()` 和 `vasprintf()` 可以用于取代 `sprintf()` 和 `vsprintf()`。

● 这些函数为字符串分配足够大的空间以容纳包括末尾空字符在内的输出内容，并通过第一个参数返回指向它的指针。

● 这些函数都是 GNU 的扩展函数，在 C 或 POSIX 标准中并没有定义。

● *BSD 系统也支持这些函数。

3、ISO/IEC WDTR 24731

具有增强的安全性的函数：fprintf_s(), printf_s(), snprintf_s(), sprintf(), vfprintf_s(), vprintf_s(), vsnprintf_s(), vsprintf_s()。

这些格式化输出函数有着不带_s 后缀的原型对应物：

- 不支持格式转换指示符%n
- 并且如果指针为空的话，它们将其视作约束违例
- 格式字符串无效

无法防止格式字符串漏洞，这些漏洞使程序崩溃，或被用于查看内存内容。

4、iostream 与 stdio

C++程序员能够使用 iostream 库，这个库提供了通过流来实现输入、输出的功能。

→ 格式化输出使用 iostream 依照中级二元插入操作符<<进行实现。

→ 左操作数是待插入数据的流。

→ 右操作数则是要插入的值。

→ 格式化和标记化输入是通过提取操作符>>实现的。

→ 标准的 I/O 流 stdin、stdout 和 stderr 被 cin、cout 和 cerr 所取代。

(1) 极其不安全的 stdio 实现

```
#include <stdio.h>
```

```
int main(int argc, char * argv[]) {
```

```
    char filename[256];  
    FILE *f;  
    char format[256];
```

从stdin读入一个文件名并尝试打开该文件。

```
    fscanf(stdin, "%s", filename);  
    f = fopen(filename, "r"); /* read only */
```

```
    if (f == NULL) {  
        sprintf(format, "Error opening file %s\n",  
                filename);  
        fprintf(stderr, format);  
        exit(-1);  
    }  
    fclose(f);  
}
```

如果打开失败的话会在第10行打印一条错误信息。

```
#include <stdio.h>
```

```
int main(int argc, char * argv[]) {
```

```
    char filename[256];  
    FILE *f;  
    char format[256];
```

这个程序容易造成缓冲区溢出

```
    fscanf(stdin, "%s", filename);  
    f = fopen(filename, "r"); /* read only */
```

```
    if (f == NULL) {  
        sprintf(format, "Error opening file %s\n",  
                filename);  
        fprintf(stderr, format);  
        exit(-1);  
    }  
    fclose(f);  
}
```

格式字符串则可能会被利用

(2) 安全的 iostream 实现

```

#include <iostream>
#include <fstream>
using namespace std;

int main(int argc, char * argv[]) {
    string filename;
    ifstream ifs;

    cin >> filename;
    ifs.open(filename.c_str());
    if (ifs.fail()) {
        cerr << "Error opening " << filename
              << endl;
        exit(-1);
    }
    ifs.close();
}

```

5、测试

- 很难构建一个能够覆盖所有路径的测试套件。
- 格式字符串 bug 的主要来源在于错误报告代码。
- 由于这类代码是作为异常的结果而被触发执行的，因此，在实际的运行期测试中这些路径往往被遗漏了。
- GNU C 编译器的选项包括 `-Wformat`，`-Wformatnonliteral`，和 `-Wformat-security`。

(1) `-Wformat`

- `-Wformat` 选项包含在 `-Wall` 选项中。
- 这个选项指示 GCC 编译器：

→ 检查对格式化输出函数的调用

→ 检查格式字符串

→ 验证所提供的参数的类型和数目都是正确的

● 不会报告带符号/无符号整数转换指示符与它们相应的参数之间不匹配的情况。

(2) Wformat-nonliteral

● 这个选项与 -Wformat 执行的功能基本相同。

● 在格式字符串不是字符串字面量并且不能被校验的情况下增加了警告功能。

● 不过当格式化函数的格式参数为 `va_list` 时除外。

(3) Wformat-security

● 这个标志执行的功能与 -Wformat 也基本相同，但它增加了对可能造成安全问题的格式化输出函数调用的警告功能。

● 在这种情况下，当格式字符串不是字符串字面量或者没有任何格式参数（如 `printf(foo)`）时就会对 `printf()` 的调用发出警告。

● 实际上，该警告是 -Wformat-nonliteral 警告的一个子集。将来可能会对 -Wformat-security 继续扩充，而新扩充的警告将不包括在 -Wformat-nonliteral 中。

(4) 词法分析

● pscan 工具是一种词法分析工具，可以自动扫描源代码中存在的格式字符串漏洞。

● 它以如下规则扫描格式化输出函数：如果函数最后一个参数是格式字符串且不是静态字符串，则产生一个报告。

- 无法侦测传入参数时存在的漏洞。

- 它在使用用户或者其他非信任源提供的格式字符串时会产生错误的判断。

- 词法分析工具最主要的优势在于速度。

- 由于词法分析工具缺乏语义知识，导致很多漏洞无法得到检测。

(5) 静态污点分析

- Shankar 描述了一个用于检测 C 程序中的格式字符串安全漏洞的系统，该系统使用了一个基于约束的类型推断引擎。

- 来自非信任源的输入会被标记为污点。

- 而由污点源衍生的数据同样会被标记为污点。

- 对于那些试图将污点数据解释为格式字符串的操作会产生一个警告。

- 这个工具是基于 cqual 扩展类型修饰符框架而构建的。

- 污点化利用附加类型修饰符来扩展现有的 C 类型系统。

- 标准 C 类型系统中已经包含了 `const` 之类的修饰符。

- 增加一个污点修饰符则允许在使用非信任输入的同时将其标记为污点。

- 例如：

```
tainted int getchar();
```

```
int main(int argc, tainted char *argv[])
```


● `getchar()` 的返回值和程序的命令行参数都被标记为污点并作为污点值对待。

● 在给定一套初始污点标注的情况下，就可以推断程序变量能否被赋予来自某个污点源的值。

● 如果任何污点类型的表达式被用作了格式字符串，则用户将会被警告程序中存在潜在的漏洞。

6、调整变参函数的实现

● 对格式字符串漏洞的利用要求参数指针被步进超过传递给格式化输出函数的参数数目。

● 格式字符串使用的参数个数超过了实际传入的实参数量。

● 可以通过将变参函数的参数个数限制在实际传递的参数个数之内，从而消除这个基于参数指针步进而导致的利用。

● 通过传递一个终止参数来判断实参什么时候被用尽是不可能的。

● ANSI 变参函数机制允许任意数据作为参数传递。

● 传递参数的数量给可变函数作为参数。

7、安全的变参函数实现

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

```

va_start()宏被展开以初始化存储变参个数的变量va_count

```

2. #define va_arg(ap,t) \
   (*(t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

```

```

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails
7.     return 0;
8. }

```

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

```

```

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails

```

展开了va_arg()宏，每次调用它的时候变量va_count都会减1。

```

7.     return 0;
8. }

```

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *) ((ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

```

```

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8);    // fails

```

当va_count等于零时如果需要参数，则函数失败。

```

7.     return 0;
8. }

```

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }

```

第一次调用**average()**的时候，由于函数选用了参数为-1作为终止条件，所以执行成功。

```

1. #define va_start(ap,v)
   (ap=(va_list)_ADDRESSOF(v)+_INTSIZEOF(v)); \
   int va_count = va_arg(ap, int)

2. #define va_arg(ap,t) \
   (*(t *)((_ap+=_INTSIZEOF(t))-_INTSIZEOF(t))); \
   if (va_count-- == 0) abort();

3. int main(int argc, char * argv[]) {
4.     int av = -1;

5.     av = average(5, 6, 7, 8, -1); // works
6.     av = average(5, 6, 7, 8); // fails

7.     return 0;
8. }

```

当**va_arg()**函数用尽所有的参数时就非正常终止了。

失败：用户忘记将-1作为参数传给函数

8、安全的变参函数绑定

- `av = average(5, 6, 7, 8); // fails`
- 1. `push 8`
- 2. `push 7`
- 3. `push 6`
- 4. `push 4 // 4 var args (and 1 fixed)`
- 5. `push 5`
- 6. `call average`
- 7. `add esp, 14h`
- 8. `mov dword ptr [av], eax`

包含变参个数的附加参数被插入到第4行。

一个汇编语言指令的例子，这是第6行调用**average()**函数所需生成的指令，以便处理修改后的变参函数实现。

(1) Exec Shield

● Exec Shield 的开发者是 Arjan van de Ven 和 Ingo Molnar，这是一种针对 Linux IA-32 的、基于核心的安全特征。

● 在 Red Hat Enterprise Linux V.3 及其更新版中，Exec Shield 能够对栈、共享库的位置以及程序堆的起点进行随机化处理。

● Exec Shield 栈随机化是由核心作为一个执行程序的启动而实现的。

● 栈指针增加一个随机值。这个过程中不会发生浪费内存的情况，因为被忽略的栈区域没有被换页载入。

(2) FormatGuard

● FormatGuard 通过插入代码实现动态检测，并且拒绝那些参数个数与转换规范所指定个数不匹配的格式化输出函数调用。

● 为了实现检查工作，应用程序必须使用 FormatGuard 进行重编译。

● FormatGuard 使用 GNU C 的预处理器 (CPP) 提取实际的参数个数，然后这个总数被传递给一个安全的包装器函数。

● 该函数解析格式字符串以确定需要传入多少个参数。

● 如果格式字符串使用的参数大于提供的实参，那么包装器函数将引发一个警报并终止进程。

● 如果攻击者使用的格式字符串能够匹配格式化输出函数的实参个数，那么 FormatGuard 将不能察觉到这种攻击。

● 攻击者有可能通过创造性地输入一些参数来发动攻击。

(3) Libsafe

● Libsafe 2.0 可以阻止企图覆写栈返回地址的格式字符串漏洞和利用。

● 如果发现了这样的攻击，Libsafe 将会记录一条警告信息并终止目标进程。

● Libsafe 包括这些有漏洞函数的更安全的版本。

● 它的首要任务就是保证这些函数在提供给它的参数的前提下能够安全工作。

● Libsafe 就调用原来的函数或者执行功能上相同的代码（例如用 `snprintf()` 代替 `sprintf()`）。

● Libsafe 以共享库的形式实现并且先于标准库载入内存。

● 某些函数被重新实现以加上必要的检查。

● 例如：为了加上两个必要的检查，重新实现了格式化输出函数。

● 第一个检查了 %n 对应的参数是否引用了一个栈帧所引用的地址。

● 第二个检查确定参数的初始地址是否和最终地址位于同一栈帧内。

(4) 静态二进制分析

● 按照如下标准，可以通过分析二进制映像来发现格式化字符串漏洞：

→ 栈修正是否比最小值还小？

→ 格式化字符串是静态的还是可变的？

● printf() 函数应该接受最少 2 个参数：一个格式化字符串和一个参数。

● 如果 printf() 只有一个参数并且该参数可变，那么这个调用也许就表示有可利用的漏洞存在。

● 传递给可变参数函数的参数个数可以通过检查函数的参数修正值进行确定。

● 例如：由于栈修正是 4，很明显只有一个参数被传递给了 printf()

```
lea eax, [ebp+10h]
```

```
push eax
```

```
call printf
```

```
add esp, 4
```

七、著名的漏洞

Wu-ftp

CDE ToolTalk

八、总结

● 对于 C99 标准格式化输出函数的不当使用可能会导致从信息漏洞乃至执行任意代码的利用。

● 格式字符串漏洞相对容易发现并被改正（例如使用 GCC 中的 `-Wformat-nonliteral` 选项）。

● 格式字符串漏洞比简单的缓冲区溢出更难利用，因为它需要同步多个指针和计数器。

● 攻击者必须对“用于查看任意位置的内存”的参数指针以及输出计数器保持追踪。

● 如果将被查看或覆写的内存地址包含空字节的话，对于利用来说可能是一个不可逾越的障碍。

● 因为格式字符串是一个“字符串”，所以当遇到第一个空字节时格式化输出函数即会退出。

● 在 Visual C++ .NET 的默认配置中，将栈放在低位内存（例如 `0x00hhhhh`）。

● 这些地址较难遭受用基于字符串操作的攻击。

● 为了消除格式字符串漏洞，推荐在可能的情况下使用 `iostream` 代替 `stdio`，在没有条件的情况下则要尽量使用静态格式字符串。

● 当需要动态字符串的时候，最关键的是不要将来自非信任源的输入合并到格式字符串中。